Not All Contexts Are Equal: Type-Aware Hybrid Context Compression for Repository-Level Code Generation

Anonymous Author(s)

Abstract

Large language models (LLMs) have demonstrated promising capabilities in repository-level code generation. However, the extensive retrieved context required for such tasks introduces substantial token overhead, increasing token consumption and the cost of API invoking. Effective context compression is therefore essential for improving the efficiency of repository-level code generation. Existing approaches often treat retrieved code uniformly, overlooking the distinct roles of different types of context in code generation. To address this limitation, we propose RepoComp, a context compression approach that applies tailored strategies to dependency context (e.g., invoked functions) and similarity context (e.g., functionally similar code). Specifically, RepoComp employs Dependency-Aware Summarization (DAS) to generate concise functional summaries for dependency context, and Task-Aware Extraction (TAE) to selectively retain implementation fragments relevant to the target task for similarity context. We evaluate RepoComp on two widely used benchmarks (i.e., DevEval and RepoExec), using three LLMs (i.e., DeepSeek-Coder-33B, Qwen2.5-Coder-32B, and DeepSeek-V4.1-Flash). Experimental results show that RepoComp reduces retrieved context tokens by over 55% while outperforming existing compression baselines in generation quality. Compared with the code-specific approach, i.e., CodePromptZip, RepoComp achieves average relative improvements of 10.63% and 18.97% in Pass@5 on DevEval and RepoExec, respectively, demonstrating the effectiveness of context-specific compression. The code is available at https://anonymous.4open.science/r/RepoComp_reproduce-8FEB.

CCS Concepts

• **Information systems** → **Information retrieval**; • **Computing methodologies** → *Natural language processing*.

Keywords

Repository-Level Code Generation, Retrieval-Augmented Code Generation, Context Compression, Large Language Models

ACM Reference Format:

Anonymous Author(s). 2018. Not All Contexts Are Equal: Type-Aware Hybrid Context Compression for Repository-Level Code Generation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In recent years, large language models (LLMs) have demonstrated remarkable capabilities across various software engineering tasks, becoming an increasingly important foundation for intelligent software development [13, 47]. Meanwhile, Web-based code hosting platforms such as GitHub have made large-scale software repositories widely accessible, providing rich sources of code knowledge for retrieval-augmented AI applications. As LLM-based applications extend from function-level tasks to real-world software repositories, models must effectively utilize cross-file contextual information, such as interface definitions, dependencies, and similar implementations distributed across repositories [6, 26].

However, the vast amount of code hosted in software repositories makes it impractical to provide an entire repository as input to an LLM. To address this challenge, retrieval-augmented code generation (RACG) has emerged as a widely adopted paradigm for repository-level code generation [46]. As a domain-specific application of retrieval-augmented generation (RAG), RACG retrieves task-relevant code from repository and incorporates it as additional context to guide code generation. Nevertheless, as the amount of retrieved context increases, token consumption and the cost of invoking commercial LLM APIs also rise, posing efficiency challenges for retrieval-augmented code generation. Therefore, effectively compressing retrieved context while preserving code generation quality is essential for improving the efficiency and cost-effectiveness of retrieval-augmented AI applications over large-scale software repositories [34].

In natural language processing (NLP), numerous general-purpose context compression approaches have been proposed, which can be broadly categorized into extractive compression [16, 32] and abstractive compression [44]. Specifically, extractive approaches select relevant fragments from the original context, whereas abstractive approaches summarize its essential information. However, recent studies [12, 18] have shown that these general-purpose approaches often yield suboptimal performance when applied to code context. More recently, several studies have explored code-specific context compression techniques based on the syntactic structure of source code [12, 40]. These approaches typically parse retrieved code into syntactic components, such as identifiers, operators and signatures, and identify the relative importance of these components through empirical analysis to remove less important ones. However, these approaches generally treat retrieved code context uniformly, overlooking the distinct roles that different types of context play in code generation.

For repository-level code generation, we can typically retrieve hybrid context comprising two distinct types: *dependency context* and *similarity context* [21, 25]. Dependency context provides information about program entities that the target code may rely on, such as functions it may invoke, enabling the generated code to

interact correctly with the existing repository. In contrast, similarity context provides implementations of functionally related code, serving as references for implementation patterns. Given their distinct roles, these two types of context differ substantially in the information that needs to be preserved. Consequently, a uniform compression strategy may struggle to accommodate both types effectively, motivating the need for type-specific context compression mechanisms for repository-level code generation.

Our key insight is that, although both dependency and similarity contexts provide valuable guidance for code generation, they differ fundamentally in the information they convey. When using a dependency, developers typically need to understand what functionality it provides and how to invoke it, without necessarily examining its complete implementation [29, 30]. Therefore, concise interface descriptions may be sufficient to support the correct use of dependency entities. By contrast, similar code is primarily used as an implementation reference [2], where specific code fragments, such as calls to helper functions, can provide direct guidance for implementing the target task. Therefore, dependency context is better suited to abstraction into interface descriptions, whereas similarity context should preserve concrete implementation logic that is highly relevant to the target task.

Motivated by these observations, we propose **RepoComp**, a context compression approach tailored to repository-level code generation. RepoComp consists of two strategies for diffent contexts: **Dependency-Aware Summarization (DAS)** for dependency context and **Task-Aware Extraction (TAE)** for similarity context. For dependency context, DAS aims to generate accurate and concise summaries of dependency items. Specifically, DAS first recursively traverses the dependency graph starting from the target item down to its leaf dependencies (nodes with no further dependencies). It then generates summaries in a bottom-up order, summarizing each item using its own source code alongside the previously generated summaries of its direct dependencies. By integrating contextual dependency information into the summarization process, DAS captures core functionality and interface details, replacing the complete implementations of dependencies with concise summaries to reduce token consumption. For similarity context, TAE aims to retain implementation details most relevant to the target task. Specifically, we first filter high-quality repositories from The Stack [19] and identify task-relevant lines of the retrieved similar code by measuring their similarity to the corresponding reference implementations of the target task. These labeled code lines are then used to train a task-aware extractor. During inference, the extractor predicts token-level retention scores of the similar code and use line-level voting to determine which lines to retain. In this way, TAE selectively preserves task-relevant implementation logic while removing less relevant code.

We systematically evaluate RepoComp on two widely used benchmarks, DevEval and RepoExec, using two open-source code LLMs, DeepSeek-Coder-33B [9] and Qwen2.5-Coder-32B [14], and one commercial model, DeepSeek-V4.1-Flash [5]. Experimental results show that RepoComp achieves the best code generation performance among the evaluated compression approaches while reducing retrieved context tokens by over 55% across both benchmarks and all three models. Compared with the code-specific approach, i.e., CodePromptZip [12], RepoComp achieves average relative improvements of 10.63% and 18.97% in Pass@5 on DevEval and RepoExec, respectively, while increasing the token compression ratio by 22.22 and 17.20 percentage points. Furthermore, ablation studies demonstrate the effectiveness of applying tailored compression strategies to different types of retrieved context.

Contributions. First, we analyze the distinct roles of dependency and similarity contexts in RACG, highlighting their different information preservation requirements. Second, we propose RepoComp, which employs DAS to preserve essential functional information from dependencies and TEA to retain task-relevant implementation details from similar code. Finally, extensive experiments demonstrate that RepoComp outperforms existing compression baselines in code generation performance while reducing retrieved context tokens by over 55%.

2 Background and Motivation

2.1 Task Definition

Given a query q, which contains a natural language description of the target functionality and partial code information, we assume that a retrieval system retrieves a set of relevant contexts $C = \{c_1, c_2, \ldots, c_n\}$, from the repository, where each c_i represents a code snippet, function, class, or other program entity in repository. A code generation model G takes the query q and retrieved context C as input to generate the target code $\hat{y}$:

$$\hat{y} = G(q, C). \tag{1}$$

In this work, we focus on compressing the retrieved context. Given the original retrieved context c_i, a compression method F selects, reorganizes, or summarizes the information in c_i to produce a more compact context c_i':

$$c_i' = F(q, c_i), \tag{2}$$

such that

$$|c_i'| < |c_i|, \tag{3}$$

where $|\cdot|$ denotes the number of tokens. The compressed context $C' = \{c_1', c_2', \ldots, c_n'\}$ is then used in place of the original context C to generate the target code:

$$\hat{y}' = G(q, C'). \tag{4}$$

The objective of context compression is to minimize the context length while preserving task-relevant information, ensuring that the generated code achieves performance comparable to that obtained using the original context.

2.2 Motivation

Previous studies [21, 25] have shown that code repositories can provide two types of context for code generation: *dependency context* and *similarity context*. Since these two types of context serve different purposes, applying a uniform compression strategy may fail to preserve their key information. This motivates the need for compression strategies tailored to their distinct characteristics.

Impact of Dependency Context. Dependency context primarily provides the functionality, interfaces, and usage conventions required by the target code. Therefore, retaining the complete implementations of dependencies may be unnecessary, as concise

```
# Dependent code
# A compiled regular expression for matching HTML tags, comments,
and doctype declarations, case-insensitive, multiline, and dotall.
HTML_RE = re.compile(
    r'((<([a-z]+:)?[a-z]+[^>]*/?>)(.*?(</([a-z]+:)?[a-z]+>))?|<!--.*--
>|<!doctype.*>)',
                re.IGNORECASE | re.MULTILINE | re.DOTALL)
# Generate with no context
def contains_html(input_string: str) -> bool:
    html_tag_pattern = r'<[^>]+>'
    return bool(re.search(html_tag_pattern, input_string))
# Generate with summarization of the Dependent code (yellow box)
def contains_html(input_string: str) -> bool:
    if not isinstance(input_string, str):
        raise InvalidInputError(input_string)
    return bool(HTML_RE.search(input_string))
```

Figure 1: Impact of dependency context on RACG.

descriptions of their essential functionality can provide sufficient guidance for code generation. As illustrated in Figure 1, when generating `contains_html()` without repository context, the model constructs its own regular expression to detect HTML tags (highlighted in red), failing to adequately handle cases such as HTML comments and DOCTYPE declarations. In contrast, when provided with a functional summary of the dependency (highlighted in yellow), the model correctly utilizes the existing `HTML_RE` to generate the expected implementation. Moreover, using the Qwen2.5-Coder tokenizer, the complete dependency requires 75 tokens, whereas its functional summary requires only 33 tokens. This example suggests that preserving essential functional information, rather than complete implementations, can effectively guide code generation while reducing context overhead.

Impact of Similarity Context. Similarity context primarily provides implementation guidance rather than complete code templates for direct reuse. Similar implementations within the same repository often share coding conventions and implementation patterns, such as exception handling, return value conventions, and the use of helper functions. Thus, extracting task-relevant code fragments may be sufficient to guide code generation without retaining entire implementations. As illustrated in Figure 2, when generating a function to retrieve a Dropbox path without repository context, the model simply prints an error message and returns `None` when the path cannot be found (highlighted in red), failing the tests because it does not invoke the repository-specific `error()` helper as required. In contrast, existing similar functions consistently invoke a shared `error()` helper when the target storage location cannot be found (highlighted in yellow). By providing this key error-handling logic as context, the model successfully generates the expected exception-handling code (highlighted in green). This example demonstrates that retaining task-relevant implementation fragments can reduce redundant context while guiding the model to generate code consistent with repository-specific conventions.

3 Approach

3.1 Overview

Figure 3 illustrates the overall workflow of **RepoComp**. Given the target prompt (signature and description of the target task), RepoComp first retrieves hybrid context from the repository, which is

```
# Similar code
def get_icloud_folder_location():
    # ...
    icloud_home = os.path.expanduser(yosemite_icloud_path)
    if not os.path.isdir(icloud_home):
        error(constants.ERROR_UNABLE_TO_FIND_STORAGE.format
            (provider="iCloud Drive"))
    return str(icloud_home)

def get_google_drive_folder_location():
    # ...
    if not googledrive_home:
        error(constants.ERROR_UNABLE_TO_FIND_STORAGE.format
            (provider="Google Drive install"))
    return googledrive_home
# Generate with no context
def get_dropbox_folder_location():
    # ...
    dropbox_path = data.get('path')
    if not dropbox_path:
        print("Could not find Dropbox path in info.json")
        return None
    return dropbox_path
# Generate with the context in yellow box
def get_dropbox_folder_location():
    # ...
    if not dropbox_info_path or not os.path.exists(dropbox_info_path):
        error(constants.ERROR_UNABLE_TO_FIND_STORAGE.format
            (provider="Dropbox"))
    # ...
    try:
        dropbox_home = info_json.get("personal", {}).get("path", "")
        if not dropbox_home:
            raise ValueError("Path not found in Dropbox's info.json.")
    except Exception as e:
        error(constants.ERROR_UNABLE_TO_FIND_STORAGE.format
            (provider=f"Dropbox ({e})"))
    return dropbox_home
```

Figure 2: Impact of similarity context on RACG.

categorized into two types: *dependency context*, providing required interfaces, and *similarity context*, providing reference implementations. RepoComp then applies different compression strategies to these two types of context. Specifically, for dependency context, we employ **Dependency-Aware Summarization (DAS)** to generate concise summaries for dependencies (Section 3.3). For similarity context, we adopt **Task-Aware Extraction (TAE)**, which uses a trained extractor to identify and retain implementation fragments relevant to the target task (Section 3.4). Finally, the compressed contexts are combined with the target prompt and fed into an LLM to generate the final code.

3.2 Retrieved Context

Following prior work [21], we first parse the source code in the repository into Abstract Syntax Trees (ASTs) using Python's `ast` module[1] and construct a repository-level graph consisting of nodes and edges. Each node represents a program entity, including the function, class, and global variable, while edges capture relationships between entities, such as import and usage relationships. Unlike approaches that split source code into fixed-length chunks, the representation treats complete program entities as the basic units (e.g., a function node contains its entire function body), preserving semantic integrity and providing a structured foundation [21].

[1] https://docs.python.org/3/library/ast.html

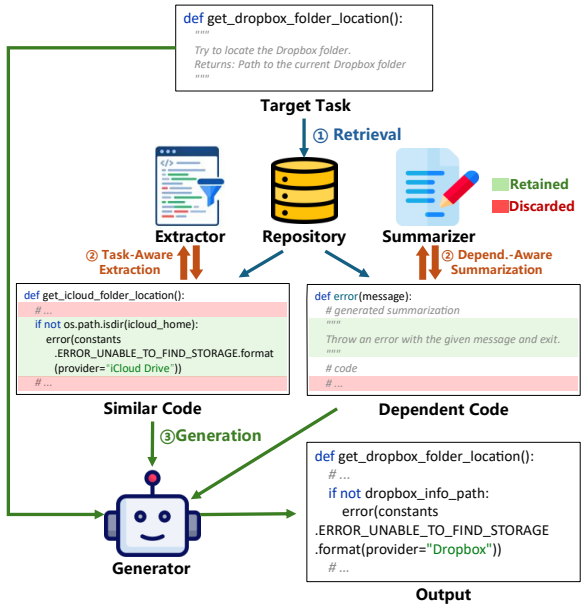


Figure 3: The overall workflow of RepoComp

Based on this representation, we retrieve hybrid context that contains two categories: *similarity context* and *dependency context*. For similarity context, following prior work [21], we employ the BM25 retriever [33] to identify the top-5 most similar program entities from the repository. For dependency context, we first collect all program entities in the file containing the target task, along with entities imported from other files in the repository. We then use the trained dependency classifier[2] to determine whether each candidate entity is relevant to the target task. Entities assigned a positive label (i.e., 1) are retained as dependency context.

3.3 Dependency-Aware Summarization

Dependency context provides program entities including functions, classes and variables that the target implementation may invoke. Previous studies [29, 30] have shown that developers typically do not need to understand the complete implementations of these entities to use them correctly. Instead, key information such as functional descriptions is often sufficient. Motivated by this observation, we replace the complete source code of dependency entities with concise functional summaries, aiming to reduce token consumption while preserving essential semantic information.

However, directly summarizing dependency code may fail to accurately capture its semantics. Unlike natural language summarization [44], program entities usually interact with the others through function calls, variable references, and other dependency relationships. Therefore, summarizing the dependency code solely based on its own source code may overlook the functionality and behavior of these external entities, resulting in inaccurate summaries.

To address this challenge, we propose **Dependency-Aware Summarization (DAS)**, which incorporates dependency information into summary generation through recursive traversal and bottom-up summarization. As illustrated in Figure 4, given a target entity, DAS first recursively traverses its dependencies until reaching leaf entities that have no further dependencies within the repository. It then generates summaries in a bottom-up order, allowing each entity to incorporate semantic information from its dependencies. Specifically, when summarizing an entity, the summarization model takes its source code and the previously generated summaries of its direct dependencies as input. This enables the model to understand not only the entity's implementation but also the functionality, interfaces, and usage constraints of the entities it relies on. By explicitly incorporating cross-entity dependency information, DAS reduces the risk of inaccurate or incomplete descriptions caused by missing external context.

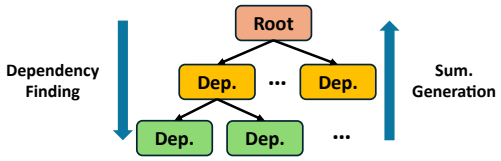


Figure 4: Recursive summarization process of DAS

Furthermore, since program entities vary in code complexity, semantic completeness, we introduce an **adaptive summary selection** to balance context length and information preservation. As illustrated in Prompt (see Appendix A.1), the summarization model selects an appropriate representation based on the entity's source code, dependency information, and existing summary, if available. Specifically, DAS handles two cases:

- **Entities without existing summaries.** If the source code is sufficiently short and its semantics are clear, DAS retains the original code to avoid unnecessary summarization and potential information loss. Otherwise, it generates a functional summary using the entity's source code and dependency information, focusing on its core functionality, interfaces, and the semantics of its parameters and return values.
- **Entities with existing summaries.** For entities that already contain descriptive information in the original repository, DAS selects or refines their representations based on the length and semantic completeness of the existing summaries. If the source code is sufficiently concise and self-explanatory, DAS uses the original code instead. Otherwise, the existing summary is retained if it is concise and semantically sufficient. Overly long summaries are shortened to remove redundant information, whereas overly brief summaries are expanded to capture missing functional details and necessary usage constraints.

This adaptive mechanism avoids applying a uniform summarization strategy to all dependency entities. Instead, DAS selects suitable representations according to their characteristics, reducing redundant implementation details while preserving information essential for code generation.

Finally, to reduce online code generation latency, we perform DAS offline by precomputing and storing summaries of repository entities. During inference, RepoComp directly retrieves these summaries without additional summarization.

[2]https://huggingface.co/solis-soict/Hydra-DAR

3.4 Task-Aware Extraction

Similarity context provides implementation guidance for the target task, such as error-handling mechanisms and the usage of helper functions. However, not all parts of similar code are relevant to the target task, making it challenging to identify the most useful implementation fragments for effective compression. Existing studies use LLMs (e.g., GPT) to construct training data for context compression [4, 44]. However, such approaches require extensive model invocations, resulting in substantial computational overhead and data construction costs.

In practice, functionally similar entities within the same repository often share common implementation patterns. For example, as illustrated in Figure 2, `get_icloud_folder_location`, `get_google_drive_location`, and the reference implementation of `get_dropbox_folder_location` exhibit highly similar error handling logic. This observation suggests that task-relevant code fragments can be identified through their similarity to the target implementation. However, the reference implementation of the target task is unavailable during inference, making direct similarity-based extraction infeasible. To address this limitation, we propose **Task-Aware Extraction (TAE)**, which leverages existing repository code to construct training data and trains a task-aware extractor.

3.4.1 Training Data Construction. We first select high-quality repositories from The Stack [19] dataset. Following prior work [43], we retain repositories containing more than five Python files, with each file having more than three import statements and at least two local imports. After filtering, we obtain 1,355 Python repositories.

Based on these repositories, we construct line-level supervision data following Algorithm 1 (see Appendix A.2). Specifically, for each function in a repository, we employ BM25 to retrieve the top five most similar program entities, forming target–similar code pairs. For each pair, we split the target and similar code into individual lines, denoted as $L^{tgt} = \{l_1^{tgt}, \ldots, l_n^{tgt}\}$ and $L^{sim} = \{l_1^{sim}, \ldots, l_m^{sim}\}$, respectively.

To identify task-relevant code lines, we compute the normalized edit similarity between each line l_i^{sim} and all lines in L^{tgt}, taking the maximum value as its relevance score:

$$s_i = \max_{1 \le j \le n} \mathrm{Sim}(l_i^{sim}, l_j^{tgt}), \tag{5}$$

where $\mathrm{Sim}(\cdot,\cdot)$ denotes the similarity measure derived from edit distance. Based on these scores, we define five similarity thresholds, $\Theta = \{0.2, 0.3, 0.4, 0.5, 0.6\}$, to construct supervision labels at different retention levels. For each threshold $\theta \in \Theta$, a line is labeled as 1 (retained) if $s_i \ge \theta$, and 0 (discarded) otherwise. Consequently, each target–similar code pair is associated with five sets of line-level supervision labels.

Finally, to ensure that the training samples contain sufficient implementation information relevant to the target task, we apply an additional filtering criterion: a target code sample is retained only if its top-ranked similar code contains at least five lines labeled as retained under the threshold $\theta = 0.2$.

3.4.2 Task-Aware Extractor Training. We train a task-aware extractor based on pretrained ModernBERT [41], formulating similarity context selection as a task-conditioned token-level binary classification problem. To accommodate different levels of code retention, we adopt a shared encoder with five independent classification heads, each trained using the supervision labels corresponding to one of the thresholds introduced in the previous section.

For each target–similar code pair, we denote the target prompt, the signatures of its dependencies, and the candidate similar code as $Tgt = (t_1, \ldots, t_k)$, $Dep = (d_1, \ldots, d_m)$, and $Cand = (c_1, \ldots, c_n)$, respectively. These components are concatenated into an input sequence $X = (Dep, Tgt, Cand)$. The target prompt specifies the generation requirements, while the dependency signatures provide information about available interfaces. Together, they guide the extractor in identifying implementation fragments within the candidate code that are relevant to the target task. Given X, the ModernBERT encoder produces contextualized representations $H = (h_1, \ldots, h_n)$ for the candidate tokens. Five independent classification heads then predict the retention probability of each candidate token under the corresponding similarity threshold:

$$p_i^{\theta} = \sigma(W_\theta h_i + b_\theta), \quad \theta \in \Theta, \tag{6}$$

where W_θ and b_θ are the trainable parameters of the corresponding classification head, and $\sigma(\cdot)$ denotes the sigmoid function.

To leverage the line-level supervision labels of training data, we assign each line's labels to all its constituent tokens. The encoder and classification heads are then jointly optimized using the binary cross-entropy loss.

3.5 Inference

During inference, RepoComp applies different compression strategies according to the type of retrieved context. For dependency context, the complete implementations are replaced with functional summaries generated by DAS, which are retrieved directly from the entity-to-summary mapping constructed during offline preprocessing. For similarity context, we adopt a dynamic threshold extraction strategy, motivated by the observation that higher-ranked candidates generally provide more relevant information. Specifically, we rank the five retrieved candidates by their BM25 scores and assign them to the classification heads corresponding to similarity thresholds of 0.2 - 0.6, respectively. Inspired by prior studies [4, 10], we aggregate token-level predictions into line-level decisions through majority voting. A code line is retained if at least half of its tokens have predicted retention probabilities greater than 0.5; otherwise, it is discarded. Finally, the dependency summaries generated by DAS and the similar code fragments extracted by TAE are combined with the target prompt to form the final prompt for generation model.

4 Experiments

4.1 Research Questions

We answer the following research questions (RQs):

RQ1: How does RepoComp perform compared to existing baselines in terms of generation quality and compression efficiency?

RQ2: How do different context types and compression strategies affect code generation performance?

RQ3: How do different dependency context representations affect compression effectiveness and code generation performance?

RQ4: How do different extraction retention ratios affect code generation performance?

Table 1: The overall performance of RepoComp and the baselines on the DevEval and RepoExec datasets. The best performance is highlighted in bold, and the second-best is underlined.

Model	Approach	DevEval					RepoExec				
		CT	τ	P@1 (%)	P@3 (%)	P@5 (%)	CT	τ	P@1 (%)	P@3 (%)	P@5 (%)
DS-Coder	No Context	0	100%	9.54	13.92	15.94	0	100%	8.76	13.06	15.75
	Full Context	5,956	0%	17.96	26.08	29.36	3,904	0%	24.92	32.03	35.24
	RECOMP	4,332	27.27%	15.16	22.41	25.91	2,833	27.41%	15.93	23.01	26.07
	LLMLingua-2	4,179	29.83%	15.60	23.40	26.91	2,784	28.67%	16.90	23.98	27.50
	CODEPROMPTZIP	3,785	36.45%	15.00	22.01	25.11	2,242	42.56%	15.01	21.66	24.64
	RepoComp	**2,587**	**56.56%**	**16.59**	**24.37**	**27.97**	**1,638**	**58.03%**	**19.65**	**27.51**	**30.66**
QW-Coder	No Context	0	100%	19.89	25.09	27.24	0	100%	15.75	19.54	21.77
	Full Context	4,620	0%	33.80	41.53	44.45	3,054	0%	37.36	44.18	46.41
	RECOMP	3,368	27.10%	28.98	36.87	40.13	2,234	26.84%	29.34	36.24	38.68
	LLMLingua-2	3,097	32.96%	28.34	36.51	39.53	2,085	31.71%	26.87	32.72	34.67
	CODEPROMPTZIP	3,114	32.60%	28.01	35.58	38.27	1,843	39.65%	22.92	29.36	32.09
	RepoComp	**2,036**	**55.92%**	**29.82**	**37.80**	**41.00**	**1,288**	**57.81%**	**31.86**	**38.76**	**41.54**
DS-V4.1-Flash	No Context	0	100%	24.20	29.40	31.60	0	100%	25.85	31.60	34.10
	Full Context	5,283	0%	40.36	47.60	50.60	3,165	0%	64.41	70.63	72.21
	RECOMP	3,786	28.32%	32.96	40.82	44.20	2,302	27.264%	56.79	63.27	65.33
	LLMLingua-2	3,596	31.94%	34.36	41.68	44.20	2,179	31.14%	54.38	61.40	64.47
	CODEPROMPTZIP	3,517	33.42%	30.92	37.82	40.40	1,902	39.90%	55.42	63.52	66.19
	RepoComp	**2,290**	**56.65%**	**37.72**	**43.68**	**45.80**	**1,333**	**57.88%**	**60.00**	**66.39**	**68.19**

4.2 Experimental Setup

Datasets. We evaluate RepoComp on two widely used benchmarks for repository-level code generation, i.e., DevEval [23] and RepoExec [11]. During evaluation, we identified several repositories with environment issues that prevented even the ground-truth implementations from passing their associated tests. To ensure reliable evaluation, we excluded the corresponding instances. After filtering, the resulting evaluation sets contain 1,505 instances from DevEval and 349 instances from RepoExec.

Baselines. To evaluate the effectiveness of RepoComp, we compare it against two categories of context compression approaches, i.e., general-purpose text compression and code-specific compression. For general-purpose approaches, we select RECOMP [44] and LLMLingua-2 [32]. For code-specific approaches, we include CodePromptZip [12].

Evaluation Metrics. We evaluate code generation quality using Pass@k [1], with $k \in \{1, 3, 5\}$. To assess compression effectiveness, we adopt Context Tokens (CT) and Compression Ratio (τ). Specifically, τ measures the percentage reduction in token count relative to the original context:

$$\tau = \left(1 - \frac{|C'|}{|C|}\right) \times 100\%, \tag{7}$$

where $|C|$ and $|C'|$ denote the number of tokens in the original and compressed retrieved contexts, respectively. A higher τ indicates greater context reduction.

Implementation Details. All experiments are conducted on a server equipped with four NVIDIA RTX 5090 GPUs. We employ two open-source models, DeepSeek-Coder-33B-Instruct (DS-Coder) [9] and Qwen2.5-Coder-32B-Instruct (QW-Coder) [14], and one commercial model, DeepSeek-V4.1-Flash (DS-V4.1-Flash) [5], for code generation. The open-source models are obtained from Hugging Face[3][4], while DeepSeek-V4.1-Flash is accessed through its API[5]. To reduce API costs, we randomly sample 500 instances from DevEval for evaluating DeepSeek-V4.1-Flash. For DAS, we use Qwen2.5-Coder-32B-Instruct to generate summaries of repository entities during offline preprocessing. For TAE, we fine-tune the pretrained ModernBERT model with a learning rate of 2×10^{-5} and a per-device batch size of 16. Due to computational time constraints, we limit extractor training to 30,000 optimization steps. Additional details on baseline configurations and other settings are provided in Appendix B.

4.3 Main Result (RQ1)

Table 1 presents the overall performance of RepoComp and the baselines on DevEval and RepoExec, evaluated in terms of code generation quality and context compression effectiveness. Across all datasets, RepoComp consistently achieves the highest Pass@1, Pass@3, and Pass@5 among compression approaches, while reducing context tokens by more than 55%. In contrast, the baselines achieve compression ratios below 43% while yielding lower code generation performance. These results demonstrate that RepoComp achieves a better trade-off between context reduction and code generation quality.

Moreover, RepoComp consistently outperforms the code-specific baseline, i.e., CodePromptZip, in both compression effectiveness

[3]https://huggingface.co/deepseek-ai/deepseek-coder-33b-instruct
[4]https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct
[5]https://www.deepseek.com/

Table 2: Impact of context types and compression strategies on generation performance. Sim. Cxt and Dep. Cxt denote using only similarity and dependency contexts, respectively. Ext. and Sum. denote applying extractive and abstractive compression to all retrieved contexts, respectively. For Ext., dependency contexts are extracted using $\theta = 0.3$.

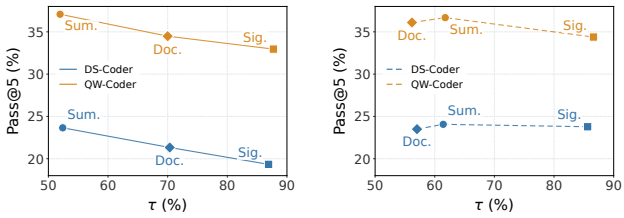
Model	context	DevEval		RepoExec	
		CT	P@5 (%)	CT	P@5 (%)
DS-Coder	No Context	0	15.94	0	15.75
	Sim. Cxt.	1,830	25.58	1,134	26.64
	Dep. Cxt.	1,291	22.99	816	25.78
	Ext.	2,479	27.84	1,800	28.08
	Sum.	1,901	23.65	1,324	24.07
	RepoComp	2,587	**27.97**	1,638	**30.66**
QW-Coder	No Context	0	27.24	0	21.77
	Sim. Cxt.	1,453	37.47	900	35.53
	Dep. Cxt.	995	34.95	629	35.24
	Ext.	1,965	38.93	1,438	36.10
	Sum.	1,467	37.07	1,021	36.67
	RepoComp	2,036	**41.00**	1,288	**41.54**

and code generation quality. On DevEval, RepoComp increases the average token compression ratio from 34.16% to 56.38%, a gain of 22.22 percentage points, while improving Pass@5 by an average of 10.63% across the three models. Similar improvements are observed on RepoExec, where RepoComp increases the average compression ratio from 40.70% to 57.91%, a gain of 17.20 percentage points, alongside an average relative Pass@5 improvement of 18.97%. These results suggest that RepoComp can preserve task-relevant information while achieving greater context reduction. For more analysis (e.g., API costs for commercial models and compression latency), please refer to Appendix C.

4.4 Impact of Context Types and Compression Strategies (RQ2)

Table 2 presents the impact of different context types and compression strategies on the performance of code generation. The results reveal two key findings.

Different context types contribute differently to code generation. Both similarity and dependency contexts substantially improve Pass@5 compared with the no-context setting, demonstrating the importance of repository-level information. However, similarity context consistently achieves higher Pass@5 than dependency context across both models and benchmarks. For instance, on DevEval, similarity context achieves Pass@5 scores of 25.58% and 37.47% with DeepSeek-Coder-33B-Instruct and Qwen2.5-Coder-32B-Instruct, respectively, outperforming dependency context by 2.59 and 2.52 percentage points. These results indicate that both types of context positively contribute to code generation performance, albeit to different extents, highlighting the importance of incorporating relevant repository context.



(a) DevEval (b) RepoExec

Figure 5: Impact of different dependency context representations on compression ratio and code generation performance.

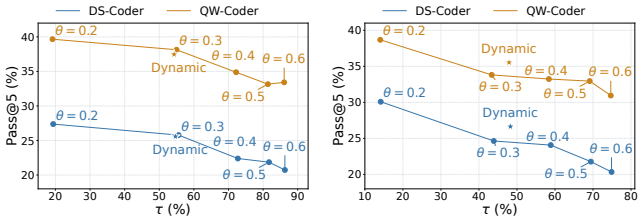
Applying tailored compression strategies yields better performance than using a uniform strategy. In terms of code generation performance, uniform extractive compression generally outperforms abstractive compression, suggesting that preserving original code fragments is often beneficial for code generation. However, abstractive compression typically produces fewer context tokens than extractive compression. Nevertheless, RepoComp consistently achieves the highest Pass@5 across all four experimental settings. Specifically, compared with uniform extractive compression, RepoComp improves Pass@5 by 0.13 and 2.07 percentage points on DevEval, and by 2.58 and 5.44 percentage points on RepoExec, using DeepSeek-Coder-33B-Instruct and Qwen2.5-Coder-32B-Instruct, respectively. These findings support our motivation that dependency and similarity contexts have different information preservation requirements, making type-specific compression more effective than applying a uniform strategy.

4.5 Impact of Dependency Context Representations (RQ3)

Figure 5 illustrates the impact of different dependency context representations on token compression ratio and code generation performance. We compare three strategies: DAS-generated functional summaries (Sum.), entity signatures only (Sig.), and original signatures combined with repository docstrings (Doc.).

On DevEval, DAS demonstrates a clear advantage in code generation performance. Compared with using entity signatures alone, DAS improves Pass@5 from 19.34% to 23.65% on DeepSeek-Coder-33B-Instruct and from 32.96% to 37.07% on Qwen2.5-Coder-32B-Instruct. Even when original repository docstrings are included, generation performance remains below that achieved by DAS. One possible explanation is that some entities in DevEval lack sufficiently informative documentation. By incorporating source code and dependency information, DAS can provide richer functional descriptions that better support code generation.

Unlike DevEval, most entities in RepoExec contain original docstrings. Nevertheless, DAS achieves better code generation performance than Doc. while using fewer context tokens. Specifically, DAS achieves compression ratios of 61.42% and 61.76% on DS-Coder and QW-Coder, respectively, compared with 57.03% and 56.18% for Doc. Meanwhile, Pass@5 improves from 23.50% to 24.07% on DS-Coder and from 36.10% to 36.68% on QW-Coder. These results suggest that DAS can further refine existing documentation to produce more concise and informative functional descriptions.



(a) DevEval (b) RepoExec

Figure 6: Impact of different extraction thresholds on generation performance and compression ratio.

Overall, these findings indicate that entity signatures alone may omit essential functional semantics, while existing repository docstrings may not always provide concise information for code generation. By incorporating dependency information into functional summaries, DAS offers a more effective balance between information preservation and context reduction. For, more experimental result of DAS, see Appendix C.5.

4.6 Impact of Extraction Thresholds (RQ4)

We adopt a dynamic threshold strategy that assigns extraction thresholds ranging from 0.2 to 0.6 based on the BM25 retrieval rankings of similar code. To further evaluate its effectiveness, we compare this strategy with the fixed thresholds, i.e., $\theta \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$, examining their impact on token compression ratio and code generation quality.

Figure 6 presents the results. As the fixed threshold increases, the token compression ratio increases, while Pass@5 generally declines. In contrast, the dynamic strategy achieves a more favorable trade-off between compression and generation quality. Notably, on RepoExec, it achieves both higher compression ratios and better Pass@5 than the fixed threshold of 0.3 across both models. On DevEval, the dynamic strategy also maintains competitive generation performance while substantially reducing context tokens. These results suggest that adapting extraction thresholds to retrieval rankings helps reduce redundant context while preserving information relevant to code generation.

5 Related Work

5.1 Retrieval-Augmented Code Generation

Code generation has long been an important research topic in software engineering [13, 17]. In recent years, large language models (LLMs) have achieved remarkable progress in code generation [7, 8, 15, 22, 24, 28, 31, 36, 38, 39, 42, 45]. However, traditional code generation tasks typically overlooking the repository context required in real-world software development. To bridge this gap, increasing attention has been directed toward repository-level code generation [11, 23], where retrieval-augmented code generation (RACG) leverages relevant repository information to guide code generation [35]. Existing studies have primarily focused on retrieving relevant repository context for the target task. For example, Repoformer [43] adopts a selective retrieval mechanism, enabling the model to determine whether external context is necessary for the current task. More recently, RepoFuse [25] and Hydra [21] have explored the roles of different types of repository context, particularly dependency and similarity contexts. Among them, Hydra employs a trained dependency classifier to identify relevant program entities, improving the effectiveness of context retrieval. Unlike these studies our work investigates the compression of retrieved context. Recognizing the distinct information requirements of dependency and similarity contexts, RepoComp employs abstractive and extractive compression strategies, respectively, to reduce context token consumption.

5.2 Retrieved Context Compression

Previous studies have shown that not all retrieved information is equally useful for a given task [4, 44]. Consequently, removing redundant or less relevant information through context compression has become an important approach to reducing the inference overhead of LLMs. Existing context compression techniques can be broadly categorized into soft compression and hard compression [37]. Soft compression methods encode the original context into compact continuous representations, which are used to convey contextual information to the generation model [3, 27]. However, these methods typically require joint training with the target generation model, limiting their applicability across different models. In contrast, hard compression methods operate directly on textual context through abstractive or extractive compression [4, 12, 32, 44]. Abstractive methods generate concise descriptions, while extractive methods retain relevant portions of the original context. By preserving textual outputs, these methods can be readily applied across different generation models.

More recently, CodePromptZip [12] analyzes the influence of different syntactic components on code generation performance and determines their retention priorities accordingly, removing less important code elements to reduce context length. However, such syntax-oriented approaches primarily focus on the importance of individual code components without explicitly distinguishing the different roles of retrieved contexts in code generation. In contrast, RepoComp considers the functional roles of retrieved context, employing dependency-aware summarization for dependency context and task-aware extraction for similarity context. This enables context-specific compression tailored to the information requirements of different types of retrieved code.

6 Conclusion

In this paper, we present RepoComp, a type-aware context compression approach for repository-level code generation. Recognizing the distinct roles of dependency and similarity contexts, RepoComp employs Dependency-Aware Summarization (DAS) to preserve essential functional information and Task-Aware Extraction (TAE) to retain task-relevant implementation fragments. Extensive experiments on DevEval and RepoExec using two code LLMs demonstrate that RepoComp reduces retrieved context tokens by over 55% while consistently outperforming existing compression baselines in code generation quality. These findings highlight the effectiveness of tailoring compression strategies to the different roles of retrieved code in repository-level code generation.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv e-prints*, Article arXiv:2107.03374 (July 2021), arXiv:2107.03374 pages. arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374

[2] Xiangping Chen, Furen Xu, Yuan Huang, Xiaocong Zhou, and Zibin Zheng. 2024. An empirical study of code reuse between GitHub and stack overflow during software development. *Journal of Systems and Software* 210 (2024), 111964. doi:10.1016/j.jss.2024.111964

[3] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. 2024. xRAG: Extreme Context Compression for Retrieval-augmented Generation with One Token. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 109487–109516. doi:10.52202/079017-3476

[4] Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. 2025. Provence: efficient and robust context pruning for retrieval-augmented generation. *arXiv e-prints*, Article arXiv:2501.16214 (Jan. 2025), arXiv:2501.16214 pages. arXiv:2501.16214 [cs.CL] doi:10.48550/arXiv.2501.16214

[5] DeepSeek-AI, :, Anyi Xu, B. Li, Bangcai Lin, Bing Xue, BingCheng Xian, Bingzheng Xu, Bochao Wu, Bowei Zhang, Boyi Deng, C. C. Yu, Chao Jin, Chaofan Lin, Chen Dong, et al. 2026. DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. *arXiv e-prints*, Article arXiv:2609.19969 (Sept. 2026), arXiv:2609.19969 pages. arXiv:2609.19969 [cs.CL] doi:10.48550/arXiv.2609.19969

[6] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, et al. 2023. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46701–46723. doi:10.52202/075280-2023

[7] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Bohua Chen, Yi Su, Dongping Chen, Siyuan Wu, Xing Zhou, Wenbin Jiang, Hai Jin, and Xiangliang Zhang. 2025. WebCode2M: A Real-World Dataset for Code Generation from Webpage Designs. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) *(WWW '25)*. Association for Computing Machinery, New York, NY, USA, 1834–1845. doi:10.1145/3696410.3714889

[8] Yi Gui, Yao Wan, Zhen Li, Zhongyi Zhang, Dongping Chen, Hongyu Zhang, Yi Su, Bohua Chen, Xing Zhou, Wenbin Jiang, and Xiangliang Zhang. 2025. UICopilot: Automating UI Synthesis via Hierarchical Code Generation from Webpage Designs. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) *(WWW '25)*. Association for Computing Machinery, New York, NY, USA, 1846–1855. doi:10.1145/3696410.3714891

[9] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv e-prints*, Article arXiv:2401.14196 (Jan. 2024), arXiv:2401.14196 pages. arXiv:2401.14196 [cs.SE] doi:10.48550/arXiv.2401.14196

[10] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjun Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to Stop? Towards Efficient Code Generation in LLMs with Excess Token Prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1073–1085. doi:10.1145/3650212.3680343

[11] Nam Le Hai, Dung Manh Nguyen, and Nghi D. Q. Bui. 2025. On the Impacts of Contexts on Repository-Level Code Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, Albuquerque, New Mexico, 1496–1524. doi:10.18653/v1/2025.findings-naacl.82

[12] Pengfei He, Shaowei Wang, and Tse-Hsun Chen. 2026. CODEPROMPTZIP: Code-specific Prompt Compression for Retrieval-Augmented Generation in Coding Tasks with LMs. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 27811–27825. doi:10.18653/v1/2026.findings-acl.1384

[13] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988

[14] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv e-prints*, Article arXiv:2409.12186 (Sept. 2024), arXiv:2409.12186 pages. arXiv:2409.12186 [cs.CL] doi:10.48550/arXiv.2409.12186

[15] Naman Jain, Tianjun Zhang, Wei-Lin Chiang, Joseph E. Gonzalez, Koushik Sen, and Ion Stoica. 2023. LLM-Assisted Code Cleaning For Training Accurate Code Generators. *arXiv e-prints*, Article arXiv:2311.14904 (Nov. 2023), arXiv:2311.14904 pages. arXiv:2311.14904 [cs.LG] doi:10.48550/arXiv.2311.14904

[16] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 13358–13376. doi:10.18653/v1/2023.emnlp-main.825

[17] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 35, 2, Article 58 (Jan. 2026), 72 pages. doi:10.1145/3747588

[18] Qiang Ke, Yanjie Zhao, Hongjin Leng, Shengming Zhao, and Haoyu Wang. 2026. Not All RAGs Are Created Equal: A Component-Wise Empirical Study for Software Engineering Tasks. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE183 (June 2026), 23 pages. https://doi.org/10.1145/3808190

[19] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The Stack: 3 TB of permissively licensed source code. *arXiv e-prints*, Article arXiv:2211.15533 (Nov. 2022), arXiv:2211.15533 pages. arXiv:2211.15533 [cs.CL] doi:10.48550/arXiv.2211.15533

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv e-prints*, Article arXiv:2309.06180 (Sept. 2023), arXiv:2309.06180 pages. arXiv:2309.06180 [cs.LG] doi:10.48550/arXiv.2309.06180

[21] Minh Le-Anh, Huyen Nguyen, An Khanh Tran, Nam Le Hai, Linh Ngo Van, Nghi D.Q. Bui, and Bach Le. 2026. Do Not Treat Code as Natural Language: Implications for Repository-Level Code Generation and Beyond. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE017 (June 2026), 24 pages. doi:10.1145/3797144

[22] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured Chain-of-Thought Prompting for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 37 (Jan. 2025), 23 pages. doi:10.1145/3690635

[23] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, et al. 2024. DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 3603–3614. doi:10.18653/v1/2024.findings-acl.214

[24] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. AceCoder: Utilizing Existing Code to Enhance Code Generation. *arXiv e-prints*, Article arXiv:2303.17780 (March 2023), arXiv:2303.17780 pages. arXiv:2303.17780 [cs.SE] doi:10.48550/arXiv.2303.17780

[25] Ming Liang, Xiaoheng Xie, Gehao Zhang, Xunjin Zheng, Peng Di, wei jiang, Hongwei Chen, Chengpeng Wang, and Gang Fan. 2024. REPOFUSE: Repository-Level Code Completion with Fused Dual Context. *arXiv e-prints*, Article arXiv:2402.14323 (Feb. 2024), arXiv:2402.14323 pages. arXiv:2402.14323 [cs.SE] doi:10.48550/arXiv.2402.14323

[26] Tianyang Liu, Canwen Xu, and Julian McAuley. 2023. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. *arXiv e-prints*, Article arXiv:2306.03091 (June 2023), arXiv:2306.03091 pages. arXiv:2306.03091 [cs.CL] doi:10.48550/arXiv.2306.03091

[27] Maxime Louis, Hervé Déjean, and Stéphane Clinchant. 2025. PISCO: Pretty Simple Compression for Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 15506–15521. doi:10.18653/v1/2025.findings-acl.800

[28] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *arXiv e-prints*, Article arXiv:2306.08568 (June 2023), arXiv:2306.08568 pages. arXiv:2306.08568 [cs.CL] doi:10.48550/arXiv.2306.08568

[29] Michael Meng, Stephanie Steinhardt, and Andreas Schubert. 2017. Application Programming Interface Documentation: What Do Software Developers Want? *Journal of Technical Writing and Communication* 48 (07 2017), 295–330. doi:10.1177/0047281617721853

[30] Michael Meng, Stephanie Steinhardt, and Andreas Schubert. 2019. How developers use API documentation: an observation study. *Commun. Des. Q. Rev* 7, 2 (Aug. 2019), 40–49. doi:10.1145/3358931.3358937

[31] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification. *arXiv e-prints*, Article arXiv:2310.10996 (Oct. 2023), arXiv:2310.10996 pages. arXiv:2310.10996 [cs.SE] doi:10.48550/arXiv.2310.10996

[32] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, et al. 2024. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 963–981. doi:10.18653/v1/2024.findings-acl.57

[33] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 4, 1-2 (09 2009), 1–174. arXiv:https://www.emerald.com/ftinr/article-pdf/4/1-2/1/11486410/1500000019en.pdf doi:10.1561/1500000019

[34] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 137 (May 2025), 22 pages. doi:10.1145/3708525

[35] Yicheng Tao, Yuante Li, Yao Qin, and Yepang Liu. 2025. Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches. *arXiv e-prints*, Article arXiv:2510.04905 (Oct. 2025), arXiv:2510.04905 pages. arXiv:2510.04905 [cs.SE] doi:10.48550/arXiv.2510.04905

[36] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2025. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1514–1526. doi:10.1109/ICSE55347.2025.00108

[37] Sourav Verma. 2024. Contextual Compression in Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv e-prints*, Article arXiv:2409.13385 (Sept. 2024), arXiv:2409.13385 pages. arXiv:2409.13385 [cs.CL] doi:10.48550/arXiv.2409.13385

[38] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F. Bissyandé, and Xiaoguang Mao. 2024. Fusing Code Searchers. *IEEE Transactions on Software Engineering* 50, 7 (2024), 1852–1866. doi:10.1109/TSE.2024.3403042

[39] Shangwen Wang, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Yan Lei, and Xiaoguang Mao. 2023. Two Birds with One Stone: Boosting Code Generation and Code Search via a Generative Adversarial Network. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 239 (Oct. 2023), 30 pages. doi:10.1145/3622815

[40] Yan Wang, Xiaoning Li, Tien N. Nguyen, Shaohua Wang, Chao Ni, and Ling Ding. 2024. Natural Is the Best: Model-Agnostic Code Simplification for Pre-trained Large Language Models. *Proc. ACM Softw. Eng.* 1, FSE, Article 27 (July 2024), 23 pages. doi:10.1145/3643753

[41] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, et al. 2025. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 2526–2547. doi:10.18653/v1/2025.acl-long.127

[42] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro von Werra, Arjun Guha, and Lingming Zhang. 2024. SelfCodeAlign: Self-Alignment for Code Generation. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 62787–62874. doi:10.52202/079017-2008

[43] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective Retrieval for Repository-Level Code Completion. *arXiv e-prints*, Article arXiv:2403.10059 (March 2024), arXiv:2403.10059 pages. arXiv:2403.10059 [cs.SE] doi:10.48550/arXiv.2403.10059

[44] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2023. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. *arXiv e-prints*, Article arXiv:2310.04408 (Oct. 2023), arXiv:2310.04408 pages. arXiv:2310.04408 [cs.CL] doi:10.48550/arXiv.2310.04408

[45] Prateek Yadav, Qing Sun, Hantian Ding, Xiaopeng Li, Dejiao Zhang, Ming Tan, Parminder Bhatia, Xiaofei Ma, Ramesh Nallapati, Murali Krishna Ramanathan, et al. 2023. Exploring Continual Learning for Code Generation Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Toronto, Canada, 782–792. doi:10.18653/v1/2023.acl-short.68

[46] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151

[47] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Shengcheng Yu, Weisong Sun, Yun Yang, and Zhenyu Chen. 2026. A survey on large language models for software engineering. *Science China Information Sciences* 69, 4 (2026), 141102. doi:10.1007/s11432-025-4670-0

A More Details of RepoComp

A.1 Prompt for DAS

Figure 7 presents the prompt template for DAS, which takes the entity signature, source code, original docstring, and summaries of its dependencies as input. The prompt guides the LLM to produce concise descriptions that preserve essential usage information, including functionality, parameters, return values, and possible exceptions. Here, `{Option}` specifies the decision rules for selecting or generating the entity representation.

As shown in Figures 8 and 9, different rules are applied depending on whether an original docstring exists. For entities without docstrings, the model either retains concise source code or generates a functional summary. For entities with docstrings, it may retain the source code, preserve the original docstring, rewrite it to remove redundancy, or expand it to include missing essential information. These decisions aim to minimize context length while preserving information necessary for correct usage.

```
Generate Python dependency interfaces for developers to use. Keep the resulting interface as short as possible while preserving the information needed for correct use. Follow the supplied rules and never invent behavior. Put the decision and text in exactly one Markdown code block labeled json.

Explain what the function or method does, its important inputs, and what it returns. Include any changes it makes, errors it may raise, or requirements the user needs to know.

{Options}

Keep the text short, but include what is needed to use the item correctly. There is no fixed sentence limit. Leave out usage examples, repetition, and implementation details. Only describe what the supplied information supports. Do not guess missing details. Mention uncertainty only when it matters for correct use.The text field is empty for a source decision; otherwise it contains plain documentation, without a signature, code fences, or surrounding docstring quotes.

Return a JSON object with exactly two fields: "decision" and "text". Wrap it in one Markdown code block, opening with ```json on its own line and closing with ``` on its own line. The code block must contain only this JSON object, with no additional fields or explanations.

Already processed child dependency interfaces:
{Dependencies}

Signature:
{Signature}

Existing docstring:
{Docstring}

Comlete source:
{Source}
```

Figure 7: Prompt template for DAS.

```
No existing docstring. Choose one:
- `use_source`: the complete docstring-free source is short and self-explanatory.
- `generate_docstring`: otherwise explain how to use the function, including what it does when the usual operation cannot be completed.
Compare the complete representation cost; do not replace clear short code with longer prose.
```

Figure 8: Options without original docstring.

```
Existing docstring. Choose one:
- `use_source`: discard the docstring only when the complete docstring-free code is short and clear, and the docstring adds nothing needed to use the function correctly.
- `keep_docstring`: copy an accurate, compact, sufficient docstring exactly.
- `rewrite_docstring`: remove verbosity, examples, or redundant member inventories. Use strictly fewer characters than the stripped original, without adding new public facts.
- `expand_docstring`: add essential supported contract missing from an incomplete docstring; it may be longer only to convey those facts. Type-only labels or vague text may be incomplete.
Rewording alone is not expansion. If a rewrite is not shorter, shorten it further or keep the original; choose expansion only when necessary supported information is added.
```

Figure 9: Options with original docstring.

Algorithm 1 Line-Level Supervision Data Construction

Require: Repositories $\mathcal{R}$, thresholds Θ, retrieval size K, minimum retained lines τ
Ensure: Supervised dataset $\mathcal{D}$

$\mathcal{D} \leftarrow \emptyset$
for each repository $R \in \mathcal{R}$ **do**
  for each function $f \in R$ **do**
    $C \leftarrow \text{BM25}(f, R, K)$
    $L^{tgt} = \{l_1^{tgt}, \ldots, l_n^{tgt}\} \leftarrow \text{SplitLines}(f)$
    $L^{top} = \{l_1^{top}, \ldots, l_p^{top}\} \leftarrow \text{SplitLines}(C[1])$
    $S^{top} = \{s_1^{top}, \ldots, s_p^{top}\} \leftarrow \text{LineSim}(L^{top}, L^{tgt})$
    if $\sum_i \mathbb{I}(S_i^{top} \geq 0.2) < \tau$ **then**
      continue
    end if
    for each candidate $c \in C$ **do**
      $L^{sim} = \{l_1^{sim}, \ldots, l_m^{sim}\} \leftarrow \text{SplitLines}(c)$
      $S = \{s_1, \ldots, s_m\} \leftarrow \text{LineSim}(L^{sim}, L^{tgt})$
      for each $\theta \in \Theta$ **do**
        $Y_\theta = \{y_1^\theta, \ldots, y_m^\theta\} \leftarrow [\mathbb{I}(s_i \geq \theta)]_{i=1}^m$
      end for
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(f, c, \{Y_\theta\}_{\theta \in \Theta})\}$
    end for
  end for
end for
return $\mathcal{D}$

A.2 Data Construction for TAE

The training procedure of the task-aware extractor is presented in Algorithm 1.

B Experimental Setup

B.1 Baseline Setup

RECOMP provides both extractive and abstractive compressors. In our preliminary experiments, we observed that its abstractive compressor discarded nearly all retrieved code, often producing empty outputs. Therefore, we adopt its extractive compressor[6].

For LLMLingua-2, we use the LLMLingua package[7] with the XLM-RoBERTa-large compressor.

CodePromptZip constructs its training data (referred to as Oracle) by progressively removing code elements according to the importance of their syntactic structures (e.g., identifiers, function signatures, and function calls) until the target compression ratio is reached. The resulting data is then used to train a compressor. However, the trained compressor, originally designed for Java, performs worse than the Oracle. Therefore, we directly adopt the Oracle-based compression results rather than training a new compressor.

Following prior work [12], we set the compression ratio threshold to 30% for each retrieved context item across all baselines.

For RECOMP and LLMLingua-2, we use the tokenizers of their respective compressors to determine whether the target compression ratio has been reached. For CodePromptZip, we use the Qwen2.5-Coder-32B-Instruct tokenizer to calculate the compression ratio after each syntactic unit is removed, stopping once the target ratio is reached.

[6]https://huggingface.co/fangyuan/nq_extractive_compressor
[7]https://github.com/microsoft/LLMLingua

Table 3: Performance under comparable compression budgets. The compression ratio threshold for all baselines is set to 50%, while RepoComp retains its original configuration.

Model	Approach	DevEval		RepoExec	
		τ	P@5 (%)	τ	P@5 (%)
DS-Coder	RECOMP	45.34%	23.74	42.93%	24.93
	LLMLingua-2	45.80%	24.00	46.29%	19.77
	CODEPROMPTZIP	**58.21%**	21.48	**65.42%**	20.34
	RepoComp	56.56%	**27.97**	58.03%	**30.66**
QW-Coder	RECOMP	45.25%	38.50	42.48%	33.81
	LLMLingua-2	48.95%	37.47	49.10%	36.97
	CODEPROMPTZIP	55.15%	35.11	**63.19%**	30.03
	RepoComp	**55.92%**	**41.00**	57.81%	**41.54**

B.2 Generation Setup

We use vLLM [20] for inference with the open-source models and directly access the commercial model through its API. For all models, we set the temperature to 0.2 and top-p to 0.95, generating five candidate solutions per task.

C Additional Experimental Results

C.1 Performance under Comparable Compression Budgets

To further evaluate performance under comparable token budgets, we increase the compression ratio threshold of all baselines from 30% to 50%, bringing their compression ratios closer to those of RepoComp. Since CodePromptZip iteratively removes syntactic units, removing a large unit may cause the actual compression ratio to exceed the target threshold substantially.

Table 3 presents the results. Compared with the original settings in Table 1, all baselines exhibit declines in Pass@5 under the higher compression threshold. Nevertheless, RepoComp substantially outperforms all baselines across both benchmarks and generation models under comparable compression budgets. Specifically, RepoComp achieves average Pass@5 improvements of 3.23 and 5.15 percentage points over the best-performing baselines on DevEval and RepoExec, respectively.

C.2 Compression Effectiveness on Passed Samples

To further assess the effectiveness of context compression and ensure that token reduction is not primarily achieved on samples that fail the tests, we examine the token compression ratios of successfully passed samples. As shown in Figure 10. RepoComp achieves compression ratios of 56.29% and 57.41% on DevEval and RepoExec, respectively, outperforming all baseline methods. These results indicate that RepoComp effectively reduces redundant context while preserving information sufficient for correct code generation.

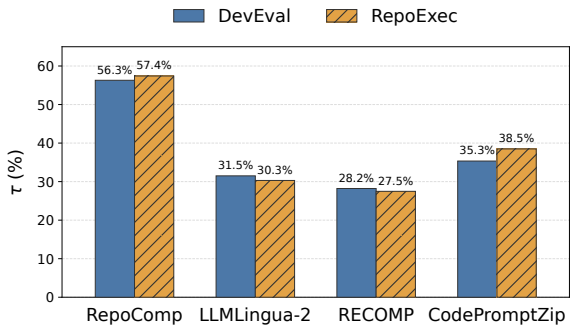


Figure 10: Token compression ratios of passed samples across different methods on DevEval and RepoExec.

Table 4: Average API token usage and total generation cost across different context compression methods. Input (I) and output (O) tokens are averaged per generation, while costs are calculated using peak pricing for all generated candidates.

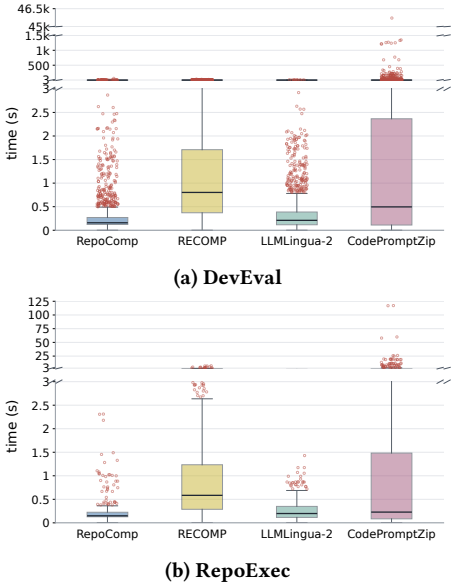
Dataset	Method	I	O	Cost ($)
DevEval	No Context	918	792	3.0655
	Full Context	6,227	898	7.3640
	LLMLingua-2	4,540	**760**	5.6847
	RECOMP	4,730	852	6.1022
	CodePromptZip	4,461	790	5.7160
	RepoComp	**3,234**	859	**5.0006**
RepoExec	No Context	136	823	1.8250
	Full Context	3,326	820	3.5172
	LLMLingua-2	2,341	**717**	2.7732
	RECOMP	2,463	777	2.9670
	CodePromptZip	2,064	719	2.6303
	RepoComp	**1,494**	738	**2.3664**

C.3 API Cost Analysis

Table 4 reports the API token usage and generation costs of different methods. RepoComp consistently achieves the lowest API cost among all compression approaches on both benchmarks. On DevEval, RepoComp reduces the total API cost by 12.03%, 18.05%, and 12.52% compared with LLMLingua-2, RECOMP, and CodePromptZip, respectively. Similarly, on RepoExec, the corresponding cost reductions are 14.67%, 20.24%, and 10.03%. These results demonstrate that RepoComp not only reduces retrieved context tokens but also effectively lowers the monetary cost of code generation.

C.4 Compression Latency

For offline preprocessing, RepoComp generates dependency summaries for all entities associated with the evaluation tasks, taking 1.75 hours on DevEval and 0.23 hours on RepoExec. During online compression, we sequentially process each retrieved context and measure the total compression time per sample. Figure 11 shows



(a) DevEval

(b) RepoExec

Figure 11: Distribution of context compression online latency

Table 5: Impact of dependency (Dep.) information on summary effectiveness.

Model	context	DevEval		RepoExec	
		CT	P@5 (%)	CT	P@5 (%)
DS-Coder	w Dep.	1,901	**23.65**	1,324	**24.06**
	w/o Dep.	1,889	22.52	1,329	23.12
QW-Coder	w Dep.	1,467	**37.07**	1,021	**36.67**
	w/o Dep.	1,455	37.00	1,024	36.39

that RepoComp and LLMLingua-2 achieve relatively low compression latency, while RECOMP and CodePromptZip are considerably slower. CodePromptZip iteratively removes code elements based on their syntactic priorities, resulting in extremely high latency for long inputs. RECOMP incurs additional overhead by computing similarities between the target prompt and individual context segments. In contrast, RepoComp and LLMLingua-2 perform compression with a single forward pass, achieving lower latency.

C.5 Impact of Dependency Information for DAS

Table 5 compares the effectiveness of summaries generated with and without dependency information. Both settings produce similar numbers of context tokens, indicating that incorporating dependency information has little impact on summary length. In terms of code generation quality, including dependency information improves Pass@5 by 1.13 and 0.07 percentage points on DevEval, and by 0.94 and 0.28 percentage points on RepoExec, using DeepSeek-Coder-33B-Instruct and Qwen2.5-Coder-32B-Instruct, respectively. These results suggest that incorporating dependency information helps generate more accurate functional summaries, thereby improving code generation performance.